1. Inverted Index & TF-IDF

Build: term -> [(docID, tf), ...] with df per term

w(t,d) = tf x log2(N/df) [on cribsheet - use log table for log10]

WHY: Fast lookup. IDF penalises common terms, rewards rare ones.

(!) EXAM TRAPS:

- df = N -> IDF = log(1) = 0 -> term contributes ZERO (e.g. "interest")
- Count tf AFTER stopword removal - recount carefully from scratch
- Equal scores = equal rank (state both, e.g. Doc1=Doc3 > Doc2)

2. Vector Space Model - Cosine Similarity

cos(D,Q) = (D . Q) / (|D| x |Q|) [on cribsheet]

Steps:

- 1. Dot product: sum d_i x q_i (only for terms in BOTH doc AND query)
- 2. |D| = sqrt(sum of ALL weights^2 in doc), |Q| = sqrt(sum of query weights^2)
- 3. Divide and round to 4dp
- Shorter docs score higher even if dot product is equal - cosine normalises length

3. Evaluation Metrics

Precision = |RelnRet|/|Ret| Recall = |RelnRet|/|Rel| F1 = 2PR/(P+R)

AP = (1/|R_total|) x sum P(k) at each relevant rank [/ TOTAL not retrieved]

DCG = rel1 + sum reli/log2(i+1) nDCG = DCG / IDCG [log2 values given]

(!) CRITICAL TRAPS:

- AP: always / TOTAL relevant - un-retrieved docs contribute 0 but still count
- From R-P graph: rank_k = k / P(at that recall point) - derive ranks this way
- Interpolated P(r) = MAX precision at recall >= r (look AHEAD in the graph)
- IDCG = ideal ranking (highest rel grades first) - then nDCG = DCG/IDCG

4. Heap's & Zipf's Laws

Heap: V = K x n^b (b=0.5 typical) -> V = K x sqrt(n)

Zipf: r x P(w|C) = A -> P(w1)=A/1, P(w2)=A/2, ...

- Heap steps: n=docsxavgdl, find K=V/sqrt(n), V_new=Kxsqrt(n_new), diff=extra t
- Larger collections -> fewer NEW terms per extra doc added (sublinear growth)
- Zipf: A = given_rank x given_prob. Top 4 words ~= 20.8% of all tokens

5. Rocchio Relevance Feedback

q_new = a.q0 + (b/|Dr|).sumDr - (g/|Dn|).sumDn [on cribsheet]

Typical: a=1 b=0.75 g=0.25 - set NEGATIVE weights to 0

Steps:

- 1. q_orig = raw TF of query string (bargain=3, books=2, etc.)
- 2. Relevant centroid = average of relevant doc vectors (/|Dr|)
- 3. Non-rel centroid = average of non-relevant doc vectors (/|Dn|)
- 4. Apply formula term by term - drop any negatives (set to 0)

"Find Similar" feature: a=0, b=1, g=0 (clicked doc becomes new query)

6. Language Models - JM Smoothing

P(w|d) = (1-L).tf(w,d)/|d| + L.P(w|C) [on cribsheet]

Score(q|d) = PRODUCT of P(w|d) for every query term w

WHY: Zero-prob fix - missing terms (tf=0) still get L.P(w|C) > 0

- Rare terms (low P(w|C)) discriminate MORE - high tf of rare term wins
- L=0.5 gives equal weight to doc frequency and collection frequency

(!) Doc with higher tf of RARER term beats doc with higher tf of common ter

7. BIM - Binary Independence Model

RSV = sum log[pi(1-si) / si(1-pi)] sum ONLY where di=1 (term present)

No relevance info: pi = 0.5 si = df / N

With relevance: pi=(ri+0.5)/(R+1) si=(df-ri+0.5)/(N-R+1)

(!) TRAPS:

- Only sum for terms where di=1 (present in doc) AND in query
- Relevant doc WITHOUT query term -> RSV=0 (big limitation of BIM)
- Non-relevant doc WITH term scores same as relevant doc - BIM cannot tell apart
- Use log10 - look up values from cribsheet log table

8. IR Model Assumptions - Always 4 marks!

- 1. Term independence (BoW): "New York" != "New" + "York" (context lost)
- 2. Document independence: ignores diversity - 10 identical docs is useless
- 3. Static relevance: ignores time - 2019 COVID doc misleading in 2026
- 4. Topical = User relevance: ignores readability, credibility, authority

-> For EACH: give the assumption AND a concrete real-world example!

9. HITS Algorithm - Hub & Authority

Hub: links TO many good authorities | Authority: linked BY many good hubs

a(p) = sum h(q) for all q->p h(p) = sum a(q) for all p->q

Matrix method (exam: "using matrices, do NOT resolve equations"):

- 1. Build A: A[i][j]=1 if page i links TO page j
- 2. Compute transpose A^T
- 3. Ma = A^TA -> write eigenvalue eqn: Ma.a = L.a (authority scores)
- 4. Mh = AA^T -> write eigenvalue eqn: Mh.h = L.h (hub scores)
- 5. Diagonal Ma[i][i] = in-degree of page i <- use this to verify your matrix!

Iterative: start h=(1,1,..), update ALL a then ALL h, normalise by max each step

10. TAAT vs DAAT Index Traversal

TAAT: Process one FULL posting list, then next. Memory: O(N) - bad for large N!

DAAT: Process all terms for ONE doc, then move to next. Memory: O(K) - good!

MaxScore / WAND (dynamic pruning):

- Precompute UB(t) = max score contribution of term t to any doc in its list
- Skip doc if partial_score + remaining_UBs < current threshold theta -> saves w

11. Index Compression - Lossy vs Lossless

Lossless: No info lost. d-gaps, VByte, Gamma codes, PForDelta

-> Measure: compression ratio + decompression speed. MAP unchanged.

Lossy: Info discarded. Static pruning, stopword removal, quantisation

-> Measure: compression ratio AND MAP/nDCG before vs after (effectiveness ma

- Acceptable if MAP drops < 1-2% with significant index size reduction

12. Exam Strategy - 1h 30min, 60 marks

- Q1 (~25 min, ~20 marks): Inverted index -> TF-IDF -> Cosine -> P/R/AP/nDCG
- Q2 (~25 min, ~20 marks): Heap's -> Rocchio -> JM Smoothing -> Conceptual Q
- Q3 (~25 min, ~20 marks): Neural IR -> PyTerrier -> 4 Assumptions or HITS

(!) ALWAYS: write formula -> substitute values -> compute -> state result = partial marks

- Check: IDF=0 trap, AP/total_relevant, normalise HITS each iteration

13. Neural IR - Architecture Trade-offs

Bi-Encoder: Encodes q and d SEPARATELY. Pre-index docs offline -> fast ANN s

- Use as FIRST STAGE - finds semantic matches (handles vocabulary mismatch)

Cross-Encoder: Encodes (q,d) JOINTLY - full interaction, best quality but slow

- Cannot pre-compute -> use as RE-RANKER on top-100 only, never first stage

ColBERT: Token-level late interaction - best of both (pre-encode docs)

MaxSim(Q,D) = sum max sim(qi,dj) over all qiinQ [on cribsheet]

(!) NaN p-value: re-ranker same 1000 candidates -> Recall@1000 identical ->

14. PyTerrier Pipelines - Up to 16 marks!

Operators: >> Then %n Top-n cutoff | Union & Intersect

PRF pipeline: bm25 % 3 >> rewriter1(index) >> bm25

- rewriter1: sees top-3 DOCS + index. Uses Bo1 / RM3 (pseudo-relevance feedbac
- rewriter2 >> bm25: sees ONLY the query string. Uses thesaurus / Word2Vec

Experiment template:

```
pt.Experiment([base, system], topics, qrels,
    eval_metrics=["ndcg_cut_10","map","recall_1000"], baseline=0)
```

- baseline=0 enables auto paired t-test. Justify: NDCG@10 = user-facing quality

15. Dense Retrieval + Passage Aggregation

(!) PROBLEM: BERT 512-token limit; single vector loses long-document cont

FIX: Split into overlapping passages, encode each independently

MaxPassage: score = BEST single passage score (one strong section proves rele

SumPassage: score = SUM of all passage scores (evidence spread throughout)

- Content SPREAD through doc? -> Use SumPassage (accumulates all evidence)
- ONE highly relevant section? -> MaxPassage is sufficient